

DACAD: Domain Adaptation Contrastive Learning for Anomaly Detection in Multivariate Time Series

Zahra Zamanzadeh Darban[✉], Yiyuan Yang[✉], Geoffrey I. Webb[✉], *Fellow, IEEE*,
Charu C. Aggarwal[✉], *Fellow, IEEE*, Qingsong Wen[✉], Shirui Pan[✉], and Mahsa Salehi

Abstract—In time series anomaly detection (TSAD), the scarcity of labeled data poses a challenge to the development of accurate models. Unsupervised domain adaptation (UDA) offers a solution by leveraging labeled data from a related domain to detect anomalies in an unlabeled target domain. However, existing UDA methods assume consistent anomalous classes across domains. To address this limitation, we propose a novel Domain Adaptation Contrastive learning model for Anomaly Detection in multivariate time series (DACAD), combining UDA with contrastive learning. DACAD utilizes an anomaly injection mechanism that enhances generalization across unseen anomalous classes, improving adaptability and robustness. Additionally, our model employs supervised contrastive loss for the source domain and self-supervised contrastive triplet loss for the target domain, ensuring comprehensive feature representation learning and domain-invariant feature extraction. Finally, an effective Center-based Entropy Classifier (CEC) accurately learns normal boundaries in the source domain. Extensive evaluations on multiple real-world datasets and a synthetic dataset highlight DACAD’s superior performance in transferring knowledge across domains and mitigating the challenge of limited labeled data in TSAD.

Index Terms—Anomaly detection, time series, deep learning, contrastive learning, self-supervised learning, domain adaptation.

I. INTRODUCTION

UNSUPERVISED Domain Adaptation (UDA) is a technique used to transfer knowledge from a labeled source domain to an unlabeled target domain. Such an approach is particularly useful when labeled data in the target domain is scarce or unavailable [1]. Deep learning methods have become the predominant approach in UDA, offering advanced capabilities and significantly improved performance compared to traditional techniques. UDA is crucial in situations where the

performance of deep models drops significantly due to the discrepancy between the data distributions in the source and target domains, a phenomenon known as domain shift [2].

Applications of UDA are diverse, ranging from image and video analysis to natural language processing and time series data analysis. However, the time series domain differs greatly from the image and text data domains for which UDA methods are well developed [3], [4]. UDA can be particularly challenging for time series analysis, given that often i) the aspects of time series that may be relevant to different tasks can vary greatly, such as frequency, magnitude, rate of change in either of these, global shape or local shape [2], [5], [6] and ii) the number of anomalous classes changes between the source and target domains.

For time series data specifically, UDA methods often employ neural architectures as feature extractors. These models are designed to handle domain adaptation, primarily target time series regression and classification problems [7], [8], [9], [10], focusing on aligning the major distributions of two domains. This approach may lead to negative transfer effects [11] on minority distributions, which is a critical concern in time series anomaly detection (TSAD). The minority distributions, often representing rare events or anomalies, may be overshadowed or incorrectly adapted due to the model’s focus on aligning dominant data distributions, potentially leading to a higher rate of false negatives in anomaly detection [12]. Since transferring knowledge of anomalies requires aligning minority distributions, anomaly label spaces often have limited similarity across domains, and existing methods face limitations in addressing anomaly detection in time series data. This highlights a significant gap in current UDA methodologies, pointing towards the need for novel approaches that can effectively address the unique requirements of anomaly detection in time series data.

To further substantiate our approach, theoretical insights indicate that reducing contrastive loss lowers the Class-wise Mean Maximum Discrepancy (CMMD) [13], thereby improving domain adaptation. This reinforces the validity of integrating contrastive learning with UDA in our approach and strengthens the theoretical foundation of our work.

Furthermore, in the realm of time series anomaly detection, in the recent model called ContextDA [14], the discriminator aligns source/target domain windows without leveraging label information in the source domain, which makes the alignment ineffective. Unlike these approaches, our proposed model leverages source labels and anomaly injection to enhance feature

Received 11 July 2024; revised 7 March 2025; accepted 10 May 2025. Date of publication 13 May 2025; date of current version 7 July 2025. Recommended for acceptance by Zhen Wang. (Corresponding author: Zahra Zamanzadeh Darban.)

Zahra Zamanzadeh Darban, Geoffrey I. Webb, and Mahsa Salehi are with the Department of Data Science and AI, Monash University, Melbourne, VIC 3800, Australia (e-mail: zahra.zamanzadeh@monash.edu; geoff.webb@monash.edu; mahsa.salehi@monash.edu).

Yiyuan Yang is with the Department of Computer Science, University of Oxford, OX1 3SA Oxford, U.K. (e-mail: yiyuan.yang@cs.ox.ac.uk).

Charu C. Aggarwal is with IBM T. J. Watson Research Center, Yorktown Heights, NY 10598 USA (e-mail: charu@us.ibm.com).

Qingsong Wen is with Squirrel AI Learning, Bellevue, WA 98004 USA (e-mail: qingsongedu@gmail.com).

Shirui Pan is with the School of ICT, Griffith University, Nathan, QLD 4111, Australia (e-mail: s.pan@griffith.edu.au).

Digital Object Identifier 10.1109/TKDE.2025.3569909

extraction, particularly by focusing on distinguishing normal from anomalous samples through contrastive learning. This anomaly injection follows the method that has been proposed in the CARLA model [15], demonstrating its efficacy in real-world scenarios. Our model leverages source labels and anomaly injection for better feature extraction, enhancing the alignment of normal samples. This is particularly vital due to the challenges in aligning anomalies across domains with different anomalous classes.

In our study, we introduce the Domain Adaptation Contrastive learning model for Anomaly Detection in time series (DACAD), a unique framework for UDA in multivariate time series (MTS) anomaly detection, which leverages contrastive learning (CL) and focuses on contextual representations. It forms positive pairs based on proximity and negative pairs using anomaly injection, subsequently learning their embeddings with both supervised CL in the source domain and self-supervised CL in the target domain. To sum up, we make the following contributions:

- We introduce DACAD, a pioneering contrastive learning framework for multivariate time series anomaly detection with UDA. DACAD is a novel deep UDA model that incorporates both supervised contrastive learning in the source domain and self-supervised contrastive learning in the target domain. To address the scarcity of real anomalies, we enhance representation learning through synthetic anomaly injection, providing effective differentiation between normal and anomalous patterns. The source code of DACAD is available on GitHub.¹
- Our proposed Center-based Entropy Classifier (CEC) introduces a novel enhancement to DeepSVDD [16] by explicitly leveraging source domain label information to refine anomaly detection. CEC enforces spatial separation in the feature space by pulling “normal” sample representations closer to the center and distancing anomalous ones. This adjustment is guided by a distance metric that enhances anomaly detection by improving class separability.
- Our comprehensive evaluation with real-world datasets highlights DACAD’s efficiency. In comparison with the recent TSAD deep models and UDA models for time series classification and UDA for anomaly detection, DACAD demonstrates superior performance, emphasizing the effective contribution of our approach.

II. RELATED WORKS

In this section, we provide a brief survey of the existing literature that intersects with our research. We concentrate on two areas: UDA for time series and deep learning for TSAD.

UDA for time series: In the realm of UDA for time series analysis, the combination of domain adaptation techniques with the unique properties of time series data presents both challenges and opportunities [17]. Traditional approaches such as Maximum Mean Discrepancy (MMD) [18] and adversarial learning frameworks [19] are geared towards minimizing domain discrepancies by developing domain-invariant features. These methods are vital in applications spanning medical, industrial,

and speech data, with significant advancements in areas like sleep classification [20], energy safety detection [21], arrhythmia classification [22], and various forms of anomaly detection [14], fault diagnosis [23], and lifetime prediction [24].

Time series-specific UDA approaches like variational recurrent adversarial deep domain adaptation (VRADA) [7], pioneered UDA for MTS, utilizing adversarial learning with an LSTM network [25] and variational RNN [26] feature extractor. Convolutional deep domain adaptation for time series (CoDATs) [10], built on VRADA’s adversarial training, but employed a convolutional neural network as the feature extractor. A metric-based method, time series sparse associative structure alignment (TS-SASA) [8] aligns intra and inter-variable attention mechanisms between domains using MMD. Adding to these, the CL for UDA of time series (CLUDA) [9] model offers a novel approach, enhancing domain adaptation capabilities in time series. All these methods share a common objective of aligning features across domains, each contributing unique strategies to the complex challenge of domain adaptation in time series classification data. However, they are ineffective when applied to TSAD tasks. Additionally, ContextDA [14] is a TSAD model that applies deep reinforcement learning to optimize domain adaptation, framing context sampling as a Markov decision process. However, it is ineffective when the anomaly classes change between the source and the target.

Recently, Large Language Models (LLMs) and Pretrained Models (PMs) have gained attention for addressing Unsupervised Domain Adaptation (UDA) in time series tasks [27], [28]. These models leverage their robust transfer learning capabilities and the benefits of extensive pre-training on a vast array of sequential data across different domains. This pre-training phase equips the models with comprehensive knowledge and advanced pattern recognition skills, allowing them to effectively adapt to and perform well on specific time series tasks through a relatively straightforward fine-tuning process [29], [30]. This extensive pre-training enables LLMs and PMs to adeptly handle time series anomaly detection, even when these tasks differ significantly from the domains on which they were originally trained. Additionally, diffusion model-based UDA methods have emerged as a promising approach, utilizing the inherent strengths of diffusion models in capturing dynamic changes over time [31]. By integrating a small amount of target domain data to guide the adaptation process, these models can achieve effective domain transfer [32], [33]. However, neither LLM nor diffusion models are without their challenges. Issues such as computational efficiency and training stability are prevalent and continue to be significant areas of research and discussion in the field [30], [31].

Deep Learning for TSAD: The field of Time Series Anomaly Detection (TSAD) has advanced significantly, embracing a variety of methods ranging from basic statistical approaches to sophisticated deep learning techniques [34], [35], [36]. Notably, deep learning has emerged as a promising approach due to its autonomous feature extraction capabilities [37]. TSAD primarily focuses on unsupervised [38], [39], [40], [41] and semi-supervised methods [42], [43], [44] to tackle the challenge of limited labeled data availability. Unsupervised methods like OmniAnomaly [45] and GDN [46] are especially valuable in

¹<https://github.com/zamanzadeh/DACAD>

scenarios with sparse anomaly labels, whereas semi-supervised methods leverage available labels effectively.

Advanced models such as LSTM-NDT [39] and THOC [47] excel in minimizing forecasting errors and capturing temporal dependencies. However, models like USAD [40] face challenges with long time series data due to error accumulation in decoding. Additionally, unsupervised representation learning, exemplified by SPIRAL [48] and TST [49], shows impressive performance, albeit with scalability issues in long time series. Newer models like TNC [50] aim to overcome these challenges using methods such as time-based negative sampling.

Furthermore, contrastive representation learning, crucial in TSAD for pattern recognition, groups similar samples together while distancing dissimilar ones [51]. It has been effectively employed in TS2Vec [38] for multi-level semantic representation and in DCdetector [52], which uses a dual attention asymmetric design for permutation invariant representations. Recently, PatchAD [53] designs a lightweight patch-based MLP-mixer based on contrastive learning for TSAD.

Moreover, time series transformers leverage self-attention mechanisms specifically adapted for sequential data, enabling them to capture both long-term dependencies and local patterns effectively. For instance, approaches like AnomalyTransformer [41] and Patch-based TS models [53] have demonstrated improved performance by focusing on multi-scale temporal features and patching mechanisms. Additionally, masked modeling for time series, especially for time series forecasting, inspired by techniques in natural language processing like masked language modeling, introduces pretext tasks that encourage better feature representation. By randomly masking parts of the time series during training and requiring the model to predict the missing values, these methods promote learning robust, domain-invariant representations [54], [55].

III. DACAD

Problem formulation: Given an unlabeled time series dataset T (target), the problem is to detect anomalies in T using a labeled time series dataset S (source) from a related domain.

In this section, we present DACAD, which uses temporal correlations ingeniously and adapts to differences between source and target domains. It starts with a labeled dataset in the source domain, featuring both normal and anomalous instances. Both real and synthetic (injected) anomalies aid in domain adaptation, thereby training in a manner that improves generalizability on a wide range of anomaly classes.

DACAD's core is CL as inspired by a recent UDA for time series work [9], which strengthens its ability to handle changes between domains by improving the feature representation learning in source and target domains. As described in Section II-I-C, in the target domain, we use a self-supervised contrastive loss [56] by forming triplets to minimize the distance between similar samples and maximize the distance between different samples. Additionally, in the source domain, we leverage label information of both anomaly and normal classes and propose an effective supervised contrastive loss [57] named supervised

Algorithm 1. DACAD($S, T, \alpha, \beta, \gamma, \lambda$).

Input: Source time series windows

$S = \{w_1^s, w_2^s, \dots, w_{|S|}^s\}$, Target time series windows

$T = \{w_1^t, w_2^t, \dots, w_{|T|}^t\}$, Loss coefficients $\alpha, \beta, \gamma, \lambda$

Output: Representation function ϕ^R , classifier ϕ^{CL} , centre c

1: Initialise $\phi^R, \phi^D, \phi^{CL}, c$

2: Split S to S_{norm} and S_{anom}

3: Create $S_{\text{inj}}, T_{\text{inj}} \triangleright$ Anomaly Injection (Section III-A)

4: Form S_{triplets} and $T_{\text{triplets}} \triangleright$ Pair Selection (Section III-B)

5: **for** each training iteration **do**

6: Compute $\phi^R(S), \phi^R(T), \phi^R(S_{\text{triplets}}), \phi^R(T_{\text{triplets}}) \triangleright$ (Section III-C)

7: Compute $\mathcal{L}_{\text{SupCont}}$ using (1) and $\phi^R(S_{\text{triplets}})$

8: Compute $\mathcal{L}_{\text{SelfCont}}$ using (2) and $\phi^R(T_{\text{triplets}})$

9: Compute $\mathcal{L}_{\text{Disc}}$ using (3), $\phi^R(S)$ and $\phi^R(T) \triangleright$ (Section III-D)

10: Compute $\mathcal{L}_{\text{Cls}}$ using (4) and $\phi^R(S) \triangleright$ (Section III-E)

11: $\mathcal{L}_{\text{DACAD}} \leftarrow$

$\alpha \cdot \mathcal{L}_{\text{SupCont}} + \beta \cdot \mathcal{L}_{\text{SelfCont}} + \gamma \cdot \mathcal{L}_{\text{Disc}} + \lambda \cdot \mathcal{L}_{\text{Cls}}$

12: Update model parameters to minimise $\mathcal{L}_{\text{DACAD}}$

13: **end for**

14: **Return** ϕ^R, ϕ^{CL}, c

mean-margin contrastive loss. A Temporal Convolutional Network (TCN) in DACAD captures temporal dependencies, generating domain-invariant features. A discriminator ensures these features are domain-agnostic, leading to consistent anomaly detection across domains. DACAD's strength lies in its dual ability to adapt to new data distributions and to distinguish between normal and anomalous patterns effectively. In DACAD, time series data is split into overlapping windows of size W_S with a stride of 1, forming detailed source (S) and target (T) datasets. Source windows are classified as normal (S_{norm}) anomalous (S_{anom}) based on anomaly presence. Fig. 1 shows the DACAD architecture, and Algorithm 1 details its steps. The following subsections explore DACAD's components and their functionalities.

A. Anomaly Injection

In the anomaly injection phase, we augment the original time series windows through a process of negative augmentation, thereby generating synthetic anomalous time series windows. This step is applied to all windows in the target domain (T) and all normal windows in the source domain (S_{norm}). We employ the anomaly injection method outlined in [15], which encompasses five distinct types of anomalies: Global, Seasonal, Trend, Shapelet, and Contextual. This procedure results in the creation of two new sets of time series windows. The first set, S_{inj} , consists of anomaly-injected windows derived from the normal samples in the source domain. The second set, T_{inj} , comprises anomaly-injected windows originating from the target domain data.

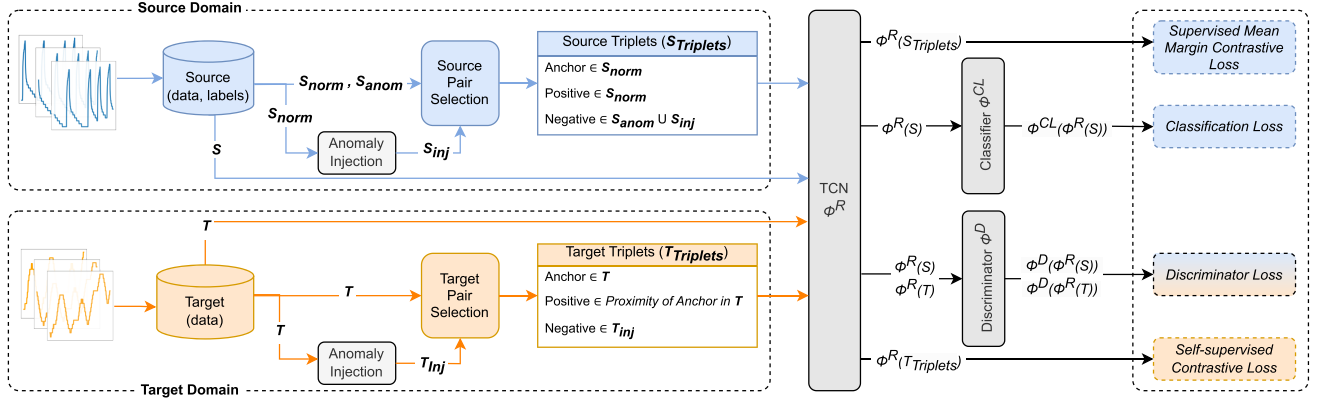


Fig. 1. DACAD Model Overview: Involves source (\$S\$) and target (\$T\$) domains. Source domain uses normal (\$S_{norm}\$) and anomalous (\$S_{anom}\$) samples, plus synthetic anomalies (\$S_{inj}\$) for source triplets (\$S_{Triplets}\$) and contrastive loss. Target domain similarly uses proximity-based pair selection and anomaly injection (\$T_{inj}\$) to create target triplets (\$T_{Triplets}\$). TCN (\$\phi^R\$) is used for feature extraction. Features from both domains are fed into the discriminator (\$\phi^D\$) for domain-invariant learning. Source features are classified by classifier \$\phi^{CL}\$.

B. Pair Selection

In DACAD's pair selection step, appropriate triplets from the source and target domains are created for CL. In the source domain, we use labels to form distinct lists of normal samples (\$S_{norm}\$), anomalous samples (\$S_{anom}\$), and anomaly-injected samples (\$S_{inj}\$). This allows for a supervised CL approach, enhancing differentiation between normal and anomalous samples. Here, triplets (\$S_{Triplets}\$) consist of an anchor (normal window from \$S_{norm}\$), a positive (different normal window from \$S_{norm}\$), and a negative (randomly selected from either an anomalous window from \$S_{anom}\$ or an anomaly-injected anchor from \$S_{inj}\$).

In the unlabeled target domain, a self-supervised approach constructs the triplets (\$T_{Triplets}\$). Each target triplet includes an anchor (original window from \$T\$), a positive (temporally close window to anchor, from \$T\$, likely sharing similar characteristics), and a negative (anomaly-injected anchor from \$T_{inj}\$).

C. Representation Layer (\$\phi^R\$)

In our model, we employ a TCN [58] for the representation layer, which is adept at handling MTS windows. This choice is motivated by the TCN's ability to capture temporal dependencies effectively, a critical aspect in time series analysis. The inputs to the TCN are the datasets and triplets from both domains, specifically \$S\$, \$T\$, \$S_{Triplets}\$, and \$T_{Triplets}\$. The outputs of the TCN, representing the transformed feature space, are

- \$\phi^R(S)\$: The representation of source windows.
- \$\phi^R(T)\$: The representation of target windows.
- \$\phi^R(S_{Triplets})\$: The representation of source triplets.
- \$\phi^R(T_{Triplets})\$: The representation of target triplets

Utilizing the outputs from the representation layer \$\phi^R\$, we compute two distinct loss functions. These are the supervised mean margin contrastive loss for source domain data (\$\mathcal{L}_{SupCont}\$) and the self-supervised contrastive triplet loss for target domain (\$\mathcal{L}_{SelfCont}\$). These loss functions are critical for training our model to differentiate between normal and anomalous patterns in both source and target domains.

1) *Supervised Mean Margin Contrastive Loss for Source Domain (\$\mathcal{L}_{SupCont}\$)*: This loss aims to embed time series windows into a feature space where normal sequences are distinctively separated from anomalous ones. It utilizes a triplet loss framework, comparing a base (anchor) window with both normal (positive) and anomalous (negative) windows [57]. Our method diverges from traditional triplet loss by focusing on the average effect of all negatives within a batch. The formula is given by (1):

$$\mathcal{L}_{SupCont} = \frac{1}{|B|} \sum_{i=1}^{|B|} \max \left(\frac{1}{|N|} \sum_{j=1}^{|N|} (\|\phi^R(a_i^s) - \phi^R(p_i^s)\|_2^2 - \|\phi^R(a_i^s) - \phi^R(n_j^s)\|_2^2 + m), 0 \right) \quad (1)$$

Here, \$|B|\$ is the batch size, and \$|N|\$ is the number of negative samples in the batch. The anchor \$a_i^s\$ is a normal time series window, and the positive pair \$p_i^s\$ is another randomly selected normal window. Negative pairs \$n_j^s\$ are either true anomalous windows or synthetically created anomalies through the anomaly injector module applied on the anchor \$a_i^s\$. This loss function uses true anomaly labels to enhance the separation between normal and anomalous behaviors by at least the margin \$m\$. It includes both genuine and injected anomalies as negatives, balancing ground truth and potential anomaly variations. The supervised mean margin contrastive loss offers several advantages for TSAD: supervised learning (uses labels for better anomaly distinction), comprehensive margin (applies a margin over average distance to negatives), and flexible negative sampling (incorporates a mix of real and injected anomalies, enhancing robustness against diverse anomalous patterns).

2) *Self-Supervised Contrastive Triplet Loss for Target Domain (\$\mathcal{L}_{SelfCont}\$)*: For the target domain, we employ a self-supervised contrastive approach using triplet loss [56], designed to ensure that the anchor window is closer to a positive window than to a negative window by a specified margin. The anchor is

assumed to be normal, following the standard TSAD assumption that the majority of the data is normal, as anomalies are rare by nature. The self-supervised triplet loss formula is shown in (2):

$$\mathcal{L}_{\text{SelfCont}} = \frac{1}{|B|} \sum_{i=1}^{|B|} \max(\|\phi^R(a_i^t) - \phi^R(p_i^t)\|_2^2 - \|\phi^R(a_i^t) - \phi^R(n_i^t)\|_2^2 + m, 0) \quad (2)$$

In this setup, the anchor window a_i^t is compared to a positive window p_i^t (a randomly selected nearby window from T) and a negative window n_i^t (the anomaly-injected version of the anchor from T_{inj}), ensuring the anchor is closer to the positive than the negative by at least the margin m .

D. Discriminator Component (ϕ^D)

The model incorporates a discriminator component within an adversarial framework. This component is crucial for ensuring that the learned features are not only relevant for the anomaly detection task but also general enough to be applicable across different domains. The discriminator is specifically trained to distinguish between representations from the source and target domains.

Designed to differentiate between features extracted from the source and target domains, the discriminator employs adversarial training techniques similar to those found in Generative Adversarial Networks [59]. Discriminator ϕ^D is trained to differentiate between source and target domain features, while ϕ^R is conditioned to produce domain-invariant features.

A crucial element is the Gradient Reversal Layer [60], which functions normally during forward passes but reverses gradient signs during backpropagation. This setup enhances the training of ϕ^D and simultaneously adjusts ϕ^R to produce features that challenge ϕ^D .

The discriminator's training involves balancing its loss against other model losses. Effective domain adaptation occurs when ϕ^R yields discriminator accuracy close to random guessing. ϕ^D , taking $\phi^R(S)$ and $\phi^R(T)$ as inputs, classifies them as belonging to the source or target domain. Its loss, a binary classification problem, minimizes classification error for source and target representations. The loss function for the discriminator ϕ^D is defined using the Binary Cross-Entropy (BCE) loss, as shown in (3):

$$\mathcal{L}_{\text{Disc}} = -\frac{1}{|S| + |T|} \left(\sum_{i=1}^{|S|} \log(f(w_i^s)) + \sum_{j=1}^{|T|} \log(1 - f(w_j^t)) \right) \quad (3)$$

where $f(w) = \phi^D(\phi^R(w))$

Here, $|S|$ and $|T|$ are the source and target window counts. w_i^s and w_j^t represent the i th and j th windows from S and T . The function $\phi^D(\phi^R(\cdot))$ computes the likelihood of a window being from the source domain.

E. Centre-Based Entropy Classifier (ϕ^{CL})

Extending the DeepSVDD [16] designed for anomaly detection, the CEC (ϕ^{CL}) in DACAD is proposed as an effective anomaly detection classifier in the source domain. It assigns anomaly scores to time series windows, using labeled data from the source domain S for training and applying the classifier to target domain data T during inference. It is centered around a Multi-Layer Perceptron (MLP) with a unique “center” parameter crucial for classification.

The classifier operates by spatially separating transformed time series window representations ($\phi^R(w_i^s)$) in the feature space relative to a learnable “center” c . The MLP aims to draw normal sample representations closer to c and push anomalous ones further away. This spatial reconfiguration is quantified using a distance metric, forming the basis for anomaly scoring. A sample closer to c is considered more normal, and vice versa. The effectiveness of this spatial adjustment is measured using a BCE-based loss function, expressed in (4):

$$\begin{aligned} \mathcal{L}_{\text{Cls}} = & -\frac{1}{|S|} \sum_{i=1}^{|S|} [y_i \cdot \log(\|g(w_i^s) - c\|_2^2) + (1 - y_i) \cdot \\ & \log(1 - \|g(w_i^s) - c\|_2^2)] \\ \text{where } g(w) = & \phi^{CL}(\phi^R(w)) \end{aligned} \quad (4)$$

In this Equation, $|S|$ is the number of samples in S , w_i^s is the i th window in S , and y_i is its ground truth label, with 1 for normal and 0 for anomalous samples.

The loss function is designed to minimize the distance between normal samples and c while maximizing it for anomalous samples. These distances are directly used as anomaly scores, offering a clear method for anomaly detection.

F. Overall Loss in DACAD

The overall loss function in the DACAD model is the amalgamation of four distinct loss components, each contributing to the model's learning process in different aspects. These components are the *Supervised Contrastive Loss* ($\mathcal{L}_{\text{SupCont}}$), the *Self-Supervised Contrastive Loss* ($\mathcal{L}_{\text{SelfCont}}$), the *Discriminator Loss* ($\mathcal{L}_{\text{Disc}}$), and the *Classifier Loss* ($\mathcal{L}_{\text{Cls}}$). The overall loss function for DACAD denoted as $\mathcal{L}_{\text{DACAD}}$, is formulated as a weighted sum of these components (with a specific weight: α for $\mathcal{L}_{\text{SupCont}}$, β for $\mathcal{L}_{\text{SelfCont}}$, γ for $\mathcal{L}_{\text{Disc}}$, and λ for $\mathcal{L}_{\text{Cls}}$), as shown in (5):

$$\mathcal{L}_{\text{DACAD}} = \alpha \cdot \mathcal{L}_{\text{SupCont}} + \beta \cdot \mathcal{L}_{\text{SelfCont}} + \gamma \cdot \mathcal{L}_{\text{Disc}} + \lambda \cdot \mathcal{L}_{\text{Cls}} \quad (5)$$

The overall loss function $\mathcal{L}_{\text{DACAD}}$ is what the model seeks to optimize during the training process. By fine-tuning these weights (α , β , γ , and λ), the model can effectively balance the significance of each loss component. This balance is crucial as it allows the model to cater to specific task requirements.

G. Role of Anomaly Injection in Convergence

Under standard assumptions in nonconvex optimization, the anomaly injection mechanism in DACAD enforces a minimum margin m between normal and anomalous embeddings. This guarantees that the contrastive losses remain strictly positive unless the model achieves the desired separation, thereby preventing degenerate solutions such as feature collapse. Assuming that the feature extractor is L -Lipschitz and that the loss functions are smooth with Lipschitz-continuous gradients [61], an update via stochastic gradient descent with a sufficiently small learning rate ensures that the overall loss is lower bounded. Moreover, under these conditions, the loss locally satisfies a Polyak–Łojasiewicz condition, which implies that the expected gradient norm decreases at a rate of $\mathcal{O}(1/K)$ over K iterations [62]. This result formally guarantees convergence to a stationary point.

H. DACAD's Inference

In the inference phase of the DACAD model, the primary objective is to identify anomalies in the target domain T . This is accomplished for each window w^t in T . The anomaly detection is based on the concept of spatial separation in the feature space, as established during the training phase. The anomaly score for each window is derived from the spatial distance between the classifier's output and a learnable center c in the feature space. This score is calculated as the squared euclidean distance, as shown in (6):

$$\text{Anomaly Score}(w^t) = \|\phi^{CL}(\phi^R(w^t)) - c\|_2^2 \quad (6)$$

Where $\phi^{CL}(\phi^R(w^t))$ denotes the feature representation of the window w^t after processing by the representation layer ϕ^R and the classifier layer ϕ^{CL} . The distance measured here quantifies how much each window in the target domain deviates from the “normal” pattern, as defined by the center c .

The anomaly score is a crucial metric in this process. A higher score suggests a significant deviation from the normative pattern, indicating a higher likelihood of the window being an anomaly. Conversely, a lower score implies that the window's representation is closer to the center, suggesting it is more likely to be normal. In practical terms, the anomaly score can be thresholded to classify windows as either normal or anomalous. By effectively utilizing these anomaly scores, the DACAD model provides a robust mechanism for identifying anomalous patterns in unlabeled target domain data.

IV. EXPERIMENTS

This section provides a comprehensive evaluation of DACAD, covering the experimental setups (Section V-A) and results (Sections V-B and V-E) to clearly understand its performance and capabilities in different contexts.

A. Datasets

We evaluate the proposed model and make comparisons across the four datasets, including the three most commonly used real benchmark datasets for TSAD and the dataset used for time series domain adaptation. The datasets are summarized

TABLE I
STATISTICS OF THE BENCHMARK DATASETS USED

Benchmark	# datasets	# dims	Train size	Test size	Anomaly ratio
MSL [39]	27	55	58,317	73,729	10.72%
SMAP [39]	55	25	140,825	444,035	13.13%
SMD [45]	28	38	708,405	708,420	4.16%
Boiler [8]	3	274	277,152	277,152	15.00%

in Table I. Since DACAD utilizes UDA, it is designed to work in scenarios where datasets contain multiple entities. Domain adaptation relies on inter-domain feature alignment, requiring at least one entity as the source and one as the target.

Mars Science Laboratory (MSL) and *Soil Moisture Active Passive (SMAP)* ² datasets [39] are real-world datasets collected from NASA spacecraft. These datasets contain anomaly information derived from reports of incident anomalies for a spacecraft monitoring system. MSL and SMAP comprise 27 and 55 datasets, respectively, and each is equipped with a predefined train/test split, where, unlike other datasets, their training set is unlabeled.

Server Machine Dataset (SMD) ³ [45] is gathered from 28 servers, incorporating 38 sensors, over a span of 10 days. During this period, normal data was observed within the initial 5 days, while anomalies were sporadically injected during the subsequent 5 days. The dataset is also equipped with a predefined train/test split, where the training data is unlabeled.

Boiler Fault Detection Dataset. ⁴ [8] The Boiler dataset includes sensor information from three separate boilers, with each boiler representing an individual domain. The objective of the learning process is to identify the malfunctioning blowdown valve in each boiler. Obtaining samples of faults is challenging due to their scarcity in the mechanical system.

V. BASELINES

Below, we will provide an enhanced description of the UDA models for time series classification and anomaly detection. Additionally, we provide a description of the five prominent and state-of-the-art TSAD models that were used for comparison with DACAD. We have selected the models from different categories of TSAD, namely, unsupervised reconstruction-based (OmniAnomaly [45] and AnomalyTransformer [41]) models, unsupervised forecasting-based (THOC [47]), and self-supervised contrastive learning (TS2Vec [38] and DCdetector [52]) TSAD models.

AE-MLP [63] is an anomaly detection model that uses autoencoders for nonlinear dimensionality reduction. It encodes data points into a lower-dimensional space and reconstructs them, highlighting deviations to detect anomalies.

AE-LSTM [64] is an anomaly detection model using an LSTM-based encoder-decoder for multi-sensor data. It detects anomalies by the reconstruction error of time series sequences.

²<https://www.kaggle.com/datasets/patrickfleith/nasa-anomaly-detection-dataset-smap-msl>

³<https://github.com/NetManAI/Ops/OmniAnomaly/tree/master/ServerMachineDataset>

⁴<https://github.com/DMIRLAB-Group/SASA-pytorch/tree/main/datasets/Boiler>

*RDC*⁵ [65] is built upon deep domain confusion to enhance domain adaptation by maximizing domain invariance. It aligns feature distributions through a domain confusion objective, making learned representations indistinguishable across domains and improving model performance on unseen domains.

*VRADA*⁶ [7] combines deep domain confusion [65] with variational recurrent adversarial deep domain adaptation [7], optimizing source domain label prediction, MMD, and domain discrimination using latent representations from the LSTM encoder. Concurrently, AE-LSTM’s reconstruction objective is utilized for anomaly detection.

*SASA*⁷ [8] improves time series classification using adaptation across domains by identifying and aligning key sparse patterns. It uses a self-attention layer with LSTM units to find optimal global context windows for source and target domains and aligns them using Maximum Mean Discrepancy (MMD).

*CLUDA*⁸ [9] is a novel UDA framework for time series data, leveraging contrastive learning to capture domain-invariant semantics while preserving label information. It is the first UDA framework designed for contextual representation learning in MTS, with effectiveness demonstrated on multiple time series classification datasets.

*ContextDA*⁹ [14] is a sophisticated approach for detecting anomalies in time series data. It formulates context sampling as a Markov decision process and employs deep reinforcement learning to optimize the domain adaptation process. This model is designed to generate domain-invariant features for better anomaly detection across various domains. It has shown promise in transferring knowledge between similar or entirely different domains.

*OmniAnomaly*¹⁰ [45] is a model operating on an unsupervised basis, employing a Variational Autoencoder (VAE) to handle MTS data. It identifies anomalies by evaluating the reconstruction likelihood of specific data windows.

*THOC*¹¹ [47] utilizes a multi-layer dilated recurrent neural network (RNN) alongside skip connections in order to handle contextual information effectively. It adopts a temporal hierarchical one-class network approach for detecting anomalies.

*TS2Vec*¹² [38] is an unsupervised model that is capable of learning multiple contextual representations of MTS and UTS semantically at various levels. This model employs contrastive learning in a hierarchical way, which provides a contextual representation. A method within TS2Vec has been proposed for application in TSAD.

*AnomalyTransformer*¹³ [41] detects anomalies in time series data by using a novel “Anomaly-Attention” mechanism to

compute association discrepancies, enhancing the distinguishability between adjacent normal and abnormal points through a minimax strategy.

*DCdetector*¹⁴ [52] is distinctive for its use of a dual attention asymmetric design combined with contrastive learning. Unlike traditional models, DCdetector does not rely on reconstruction loss for training. Instead, it utilizes pure contrastive loss to guide the learning process. This approach enables the model to learn a permutation invariant representation of time series anomalies, offering superior discrimination abilities compared to other methods.

A. Experimental Setup

In our study, we evaluate several SOTA TSAD models, including OmniAnomaly [45], TS2Vec [38], THOC [47], and DCdetector [52] and SOTA UDA models that support MTS classification including VRADA [7] and CLUDA [9] on benchmark datasets previously mentioned in Section I using their source code and best hyper-parameters as they stated to ensure a fair evaluation. The hyperparameters used in our implementation are as follows: DACAD consists of a 3-layer TCN architecture with three different channel sizes [128, 256, 512] to capture temporal dependencies. The dimension of the representation is 1024. We use the same hyperparameters across all datasets to evaluate DACAD: window size (W_S) = 100, margin $m = 1$, and we run our model for 20 epochs.

B. Baselines Comparison

Table II provides a comprehensive comparison of the performance of the different models on MTS benchmark datasets. The performance measurements include the F1 score (best F1 score), AUPR (Area Under the Precision-Recall Curve), and AUROC (Area Under the Receiver Operating Characteristic Curve). Despite Point Adjustment (PA) popularity in recent years, we do not use PA when calculating these metrics due to Siwon Kim’s findings [66] that its application leads to an overestimation of a TSAD model’s capability and can bias results in favor of methods that produce extreme anomaly scores. Instead, we use conventional performance metrics for anomaly detection.

Benchmark datasets like SMD contain multiple time series that cannot be merged due to missing timestamps, making them unsuitable for averaging their F1 scores. The F1 score is a non-additive metric combining precision and recall. To address this, we compute individual confusion matrices for each time series. These matrices are then aggregated into a collective confusion matrix for the entire dataset. From this aggregated matrix, we calculate the overall precision, recall, and F1 score, ensuring an accurate and undistorted representation of the dataset’s F1 score. Additionally, we report the Affiliation F1 (Aff-F1) [67], calculated from Aff-Precision (Aff-Pre) and Aff-Recall (Aff-Rec), to provide a comprehensive evaluation focused on early detection. For datasets with multiple time series, we present the mean and standard deviation of Aff-F1, AUPR, and AUROC for each series.

⁵<https://github.com/syorama/DDC-transfer-learning>

⁶<https://github.com/floft/vrada>

⁷<https://github.com/DMIRLAB-Group/SASA>

⁸<https://github.com/oezyurty/CLUDA>

⁹There is no implementation available for this model, so we rely on the results claimed in the paper.

¹⁰<https://github.com/smallcowbaby/OmniAnomaly>

¹¹We utilized the authors’ shared implementation, as it is not publicly available.

¹²<https://github.com/yuezhihan/ts2vec>

¹³<https://github.com/thuml/Anomaly-Transformer>

¹⁴<https://github.com/DAMO-DI-ML/KDD2023-DCdetector>

TABLE II
F1, AUPR, AND AUROC RESULTS FOR VARIOUS MODELS ON MULTIVARIATE TIME SERIES BENCHMARK DATASETS (SMD, MSL, SMAP)

Type	Model	src	SMD				MSL				SMAP			
			1-1	2-3	3-7	1-5	F-5	P-10	D-14	C-1	A-7	P-2	E-8	D-7
Classification	VRADA [7]	F1	<u>0.511</u>	0.435	0.268	0.291	0.302	0.321	0.310	0.285	0.244	0.282	0.349	0.226
		AUPR	<u>0.523±0.223</u>	0.433±0.172	0.316±0.208	0.292±0.147	0.201±0.133	0.199±0.159	<u>0.351±0.244</u>	0.167±0.132	0.133±0.196	<u>0.265±0.263</u>	0.261±0.247	0.120±0.149
		AUROC	<u>0.802±0.130</u>	0.731±0.121	0.597±0.146	0.651±0.088	0.526±0.054	0.516±0.097	0.615±0.207	0.506±0.024	0.474±0.176	<u>0.573±0.162</u>	0.612±0.120	0.443±0.170
		Aff-F1	0.840±0.128	<u>0.844±0.110</u>	<u>0.830±0.117</u>	<u>0.807±0.123</u>	0.697±0.041	0.701±0.053	0.721±0.072	0.714±0.066	0.737±0.135	<u>0.787±0.148</u>	<u>0.780±0.139</u>	<u>0.768±0.132</u>
Classification	CLUDA [9]	F1	0.435	<u>0.487</u>	0.320	0.314	<u>0.395</u>	<u>0.368</u>	<u>0.324</u>	0.312	0.292	0.278	<u>0.384</u>	0.293
		AUPR	0.423±0.223	<u>0.494±0.174</u>	<u>0.400±0.201</u>	<u>0.328±0.140</u>	<u>0.325±0.263</u>	0.239±0.1617	0.318±0.276	0.193±0.1640	<u>0.250±0.290</u>	0.242±0.217	<u>0.332±0.263</u>	<u>0.193±0.220</u>
		AUROC	0.788±0.124	<u>0.794±0.121</u>	<u>0.725±0.155</u>	<u>0.693±0.119</u>	<u>0.579±0.204</u>	0.569±0.132	0.558±0.222	0.479±0.164	<u>0.560±0.223</u>	0.500±0.209	<u>0.692±0.171</u>	0.480±0.222
		Aff-F1	0.816±0.103	0.823±0.105	0.813±0.101	0.791±0.124	0.760±0.090	0.743±0.082	0.748±0.093	0.735±0.088	0.738±0.099	0.753±0.114	0.757±0.123	0.757±0.128
Anomaly Detection	Omni [45]	F1	0.397	0.413	<u>0.419</u>	<u>0.421</u>	0.254	0.266	0.246	0.242	0.320	0.326	0.331	0.318
		AUPR	0.296±0.155	0.309±0.165	0.314±0.171	0.318±0.182	0.154±0.184	0.154±0.184	0.152±0.185	0.148±0.186	0.111±0.129	0.114±0.130	0.116±0.131	0.112±0.131
		AUROC	0.619±0.153	0.628±0.164	0.639±0.156	0.620±0.155	0.392±0.173	0.392±0.173	0.398±0.165	0.386±0.175	0.421±0.189	0.406±0.188	0.406±0.190	0.414±0.181
		Aff-F1	0.635±0.100	0.640±0.097	0.639±0.098	0.638±0.099	0.680±0.046	0.682±0.047	0.679±0.047	0.681±0.049	0.700±0.089	0.701±0.089	0.701±0.089	0.700±0.089
	THOC [47]	F1	0.156	0.167	0.168	0.168	0.307	0.317	0.309	0.310	0.321	0.328	0.335	0.320
		AUPR	0.090±0.095	0.106±0.129	0.109±0.128	0.109±0.128	0.241±0.278	<u>0.247±0.275</u>	0.244±0.277	<u>0.242±0.2778</u>	0.191±0.263	0.197±0.264	0.200±0.266	0.191±0.264
		AUROC	0.646±1.550	0.646±0.154	0.657±0.161	0.648±0.158	0.631±0.177	<u>0.637±0.182</u>	<u>0.637±0.182</u>	<u>0.640±0.183</u>	0.535±0.211	0.539±0.211	0.543±0.211	<u>0.539±0.213</u>
		Aff-F1	0.675±0.125	0.675±0.126	0.676±0.126	0.673±0.125	0.777±0.119	0.775±0.118	0.775±0.118	0.778±0.119	0.780±0.152	0.777±0.149	0.778±0.151	0.779±0.152
	TS2Vec [38]	F1	0.171	0.173	0.173	0.173	0.320	0.316	0.317	<u>0.313</u>	<u>0.362</u>	<u>0.368</u>	0.374	<u>0.362</u>
		AUPR	0.112±0.076	0.112±0.076	0.116±0.075	0.116±0.075	0.137±0.137	0.138±0.136	0.135±0.138	0.133±0.138	0.145±0.167	0.147±0.167	0.149±0.168	0.143±0.165
		AUROC	0.492±0.046	0.493±0.045	0.489±0.044	0.487±0.040	0.511±0.096	0.514±0.096	0.514±0.096	0.513±0.096	0.504±0.088	0.506±0.088	0.507±0.089	0.507±0.089
		Aff-F1	0.748±0.075	0.749±0.075	0.747±0.071	0.748±0.072	0.695±0.039	0.698±0.043	0.694±0.037	0.694±0.038	0.691±0.138	0.693±0.139	0.493±0.139	0.694±0.140
	AnomTran [41]	F1	0.295	0.307	0.304	0.306	0.345	0.347	0.343	0.346	0.402	0.402	0.409	0.398
		AUPR	0.270±0.235	0.280±0.233	0.272±0.236	0.280±0.233	0.240±0.240	0.243±0.239	0.224±0.232	0.236±0.241	0.263±0.318	0.252±0.305	0.267±0.317	0.252±0.305
		AUROC	0.761±0.142	0.760±0.142	0.768±0.142	0.759±0.140	0.633±0.142	0.637±0.147	0.631±0.140	0.641±0.148	0.654±0.195	0.647±0.190	0.652±0.195	0.648±0.191
		Aff-F1	0.659±0.241	0.660±0.242	0.665±0.240	0.663±0.239	<u>0.770±0.106</u>	<u>0.768±0.103</u>	<u>0.772±0.103</u>	<u>0.772±0.104</u>	<u>0.766±0.164</u>	0.764±0.162	0.765±0.163	0.765±0.163
	DCdetector [32]	F1	0.079	0.085	0.085	0.083	0.208	0.215	0.203	0.201	0.280	0.283	0.287	0.278
		AUPR	0.041±0.036	0.044±0.037	0.044±0.037	0.044±0.037	0.124±0.139	0.125±0.138	0.122±0.140	0.122±0.140	0.128±0.158	0.129±0.159	0.132±0.159	0.127±0.157
		AUROC	0.496±0.008	0.495±0.008	0.495±0.008	0.495±0.008	0.501±0.011	0.501±0.011	0.501±0.011	0.501±0.011	0.516±0.082	0.513±0.079	0.516±0.082	0.517±0.082
		Aff-F1	0.674±0.008	0.675±0.009	0.675±0.009	0.675±0.010	0.706±0.035	0.707±0.035	0.707±0.035	0.708±0.037	0.678±0.075	0.680±0.076	0.670±0.074	0.680±0.075
	DACAD	F1	0.600	0.633	0.598	0.572	0.595	0.528	0.522	0.528	0.532	0.464	0.651	0.463
		AUPR	0.528±0.245	0.605±0.196	0.565±0.211	0.535±0.219	0.554±0.268	0.448±0.280	0.475±0.304	0.514±0.273	0.483±0.288	0.447±0.270	0.550±0.270	0.388±0.270
		AUROC	0.856±0.094	0.858±0.083	0.822±0.093	0.813±0.098	0.787±0.268	0.748±0.142	0.719±0.191	0.769±0.151	0.760±0.151	0.687±0.202	0.790±0.105	0.619±0.234
		Aff-F1	<u>0.834±0.121</u>	0.845±0.122	0.848±0.121	0.845±0.116	0.814±0.123	0.785±0.119	0.804±0.125	0.810±0.125	0.785±0.133	0.798±0.143	0.799±0.149	0.798±0.149

The most optimal evaluation results are displayed in bold, while the second-best ones are indicated by underlining.

It is evident from Table II that DACAD is the best performer, consistently achieving the best results across all scenarios and metrics, as highlighted in bold. This suggests its robustness and adaptability to handling normal and anomalous representations of time series for anomaly detection. VRADA and CLUDA models often rank as the second best, with their results underlined in several instances. Other models like OmniAnomaly, THOC, TS2Vec, AnomalyTransformer, and DCdetector demonstrate more uniform performance across various metrics but generally do not reach the top performance levels seen in DACAD, VRADA, or CLUDA. Their consistent but lower performance could make them suitable for applications where top-tier accuracy is less critical.

The addition of the Affiliation F1 (Aff-F1) metric further reinforces the strength of DACAD. A high value of the affiliation metric indicates a strong overlap or alignment between the detected anomalies and the true anomalies in a time series. DACAD's superior Aff-F1 scores across all datasets underline

its efficacy in accurately identifying anomalies, highlighting its potential as a solution in time series anomaly detection.

C. UDA Comparison

Table III, adapted from [14], now includes results of CLUDA and our model in the last two columns. This table provides a comprehensive comparison of the performances of various models on the SMD and Boiler datasets, as measured by Macro F1 and AUROC scores.

To assess our model against ContextDA [14] — the only existing UDA for the TSAD model — we use the same datasets and metrics reported in ContextDA's main paper, as its code is unavailable. On the SMD dataset, ContextDA achieves an average macro F1 of 0.63 and AUROC of 0.75, whereas our model achieves a higher average macro F1 of 0.81 and AUROC of 0.86. Similarly, on the Boiler dataset, ContextDA's average macro F1 is 0.50 and AUROC 0.65, compared to our model's

TABLE III
MACRO F1/AUROC RESULTS ON SMD AND BOILER DATASET

Data	src $\mapsto$ trg	Models (Macro F1/AUROC)							
		AE-MLP	AE-LSTM	RDC	VRADA	SASA	CLUDA	ContextTDA	DACAD
SMD	1-1 $\mapsto$ 1-2	0.72 / 0.83	0.74 / 0.90	0.74 / 0.89	0.74 / 0.91	0.59 / 0.63	<u>0.75 / 0.84</u>	<u>0.75 / 0.91</u>	0.80 / 0.84
	1-1 $\mapsto$ 1-3	0.57 / 0.70	0.49 / 0.41	0.57 / 0.72	0.49 / 0.41	0.61 / 0.90	<u>0.67 / 0.77</u>	0.57 / 0.75	0.71 / 0.84
	1-1 $\mapsto$ 1-4	0.55 / 0.74	0.54 / 0.41	0.54 / 0.75	0.54 / 0.41	0.55 / 0.75	0.56 / 0.74	<u>0.59 / 0.76</u>	0.75 / 0.83
	1-1 $\mapsto$ 1-5	0.54 / 0.79	0.55 / 0.84	0.56 / 0.85	0.55 / 0.79	0.65 / <u>0.87</u>	<u>0.68 / 0.87</u>	0.66 / <u>0.87</u>	0.82 / 0.96
	1-1 $\mapsto$ 1-6	0.71 / 0.88	0.71 / 0.91	0.71 / 0.87	0.71 / <u>0.91</u>	0.44 / 0.84	0.71 / 0.77	<u>0.73 / 0.84</u>	0.89 / 0.92
	1-1 $\mapsto$ 1-7	0.48 / 0.55	0.48 / 0.50	0.49 / 0.54	0.48 / 0.50	0.31 / 0.57	<u>0.50 / 0.62</u>	<u>0.51 / 0.53</u>	0.88 / 0.91
	1-1 $\mapsto$ 1-8	0.55 / 0.57	0.53 / <u>0.70</u>	0.55 / 0.58	0.54 / 0.56	0.52 / 0.56	0.56 / 0.61	<u>0.58 / 0.58</u>	0.79 / 0.72
Boiler	1 $\mapsto$ 2	0.44 / 0.64	0.43 / 0.48	0.43 / 0.54	0.43 / 0.48	<u>0.53 / 0.88</u>	0.49 / 0.52	0.50 / 0.59	0.61 / 0.66
	1 $\mapsto$ 3	0.40 / 0.28	0.40 / 0.11	0.43 / 0.49	0.42 / 0.18	0.41 / 0.48	0.47 / <u>0.79</u>	<u>0.50 / 0.67</u>	0.74 / 0.81
	2 $\mapsto$ 1	0.39 / 0.18	0.40 / 0.21	0.40 / 0.36	0.40 / 0.15	<u>0.53 / 0.90</u>	0.47 / 0.49	0.51 / 0.66	0.58 / 0.65
	2 $\mapsto$ 3	0.40 / 0.38	0.40 / 0.20	0.45 / 0.39	0.42 / 0.21	0.41 / 0.49	0.48 / 0.50	<u>0.50 / 0.69</u>	0.59 / 0.63
	3 $\mapsto$ 1	0.39 / 0.20	0.40 / 0.16	0.39 / 0.31	0.40 / 0.15	0.48 / 0.67	0.47 / <u>0.77</u>	<u>0.51 / 0.67</u>	0.72 / 0.84
	3 $\mapsto$ 2	0.48 / 0.54	0.49 / 0.48	0.49 / 0.55	0.49 / 0.48	0.46 / 0.31	0.49 / <u>0.57</u>	<u>0.50 / 0.57</u>	0.58 / 0.68

The most optimal evaluation results are displayed in bold, while the second-best ones are indicated by underline.

superior performance with an average macro F1 of 0.63 and AUROC of 0.71. In this comparative analysis, the DACAD model distinctly outperforms others, consistently achieving the highest scores in both Macro F1 and AUROC across the majority of test cases in both datasets. Its performance is not only superior but also remarkably stable across different scenarios within each dataset, showcasing its robustness and adaptability to diverse data challenges.

The ContextTDA model frequently ranks as the runner-up after DACAD, particularly in the SMD dataset, where it often secures the second-highest scores in either or both the Macro F1 and AUROC metrics. Interestingly, certain models exhibit a degree of specialization. For example, SASA, which underperforms in the SMD dataset, demonstrates notably better results in specific scenarios within the Boiler dataset, particularly in terms of the Macro F1 score. This suggests that some models may be more suited to specific types of data or scenarios.

In conclusion, while the DACAD model emerges as the most effective across most scenarios, the varying performances of different models across scenarios and datasets highlight the importance of carefully evaluating each model's unique characteristics and capabilities. A thoughtful selection process is essential to ensuring the most suitable model is chosen for a given task. This subtle approach is crucial in leveraging each model's strengths and achieving optimal results.

D. Scalability Test

Scalability tests are crucial to evaluate a model's performance as data scales up in size and complexity. The plots in Fig. 2 provide a comprehensive comparison of DACAD's scalability with other models described in Section V-B. The experiment measures execution time and peak memory usage in relation to the length of the time series and their dimensionality. For the dimensionality test, eight synthetic time series datasets with varying dimensions (ranging from 8 to 1024, doubling each step) and a fixed length of 2,000 are generated using the NeurIPS-TS [68]. Another set of eight synthetic datasets with a fixed dimension of 8 varying lengths (ranging from 2,000 to 256,000) is created for the scale-up test concerning length. We report the execution time and memory usage of all models with the same parameter settings as in Section V-B.

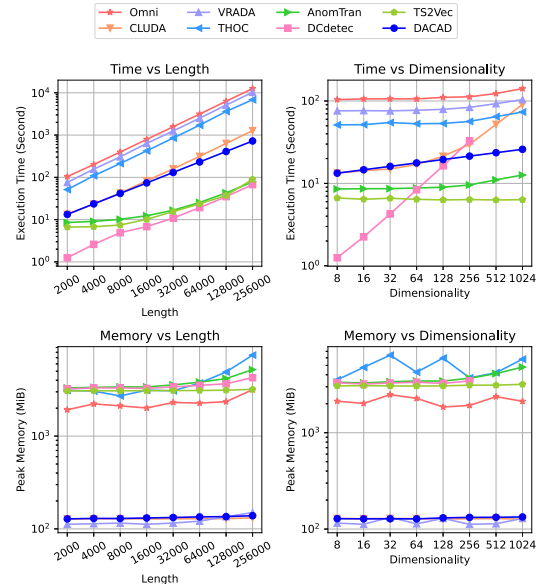


Fig. 2. Comparison of execution time and peak memory usage for DACAD and other models. The top plots show execution time against time series length and dimensionality, while the bottom plots show memory usage.

DACAD demonstrates excellent scalability in terms of time series length, outperforming other domain adaptation models such as VRADA and CLUDA. It maintains lower execution times, particularly evident with larger datasets, highlighting its efficiency. DACAD's scalability regarding time series dimensions is also remarkable. It remains stable as dimensionality increases, with execution times growing at a very small slope, ensuring it never exceeds reasonable memory limits. Compared to all other anomaly detection models, DACAD shows good scalability concerning time series length, consistently outperforming other methods and demonstrating its robustness in various performance metrics.

E. Ablation Study

Our ablation study focused on the following aspects: (1) *Effect of the loss components*, (2) *Effect of CEC*, and (3) *Effect of anomaly injection*.

Effect of the loss components:

TABLE IV
EFFECT OF LOSS COMPONENTS ON MSL DATASET (SOURCE: F-5)

Component	F1	AUPR	AUROC
DACAD	0.595	0.554±0.268	0.787±0.268
w/o $\mathcal{L}_{SelfCont}$	0.481	0.495±0.291	0.697±0.203
w/o $\mathcal{L}_{SupCont}$	0.463	0.427±0.274	0.639±0.198
w/o $\mathcal{L}_{Disc}$	0.471	0.484±0.315	0.699±0.229
w/o $\mathcal{L}_{Cls}$	0.299	0.170±0.127	0.503±0.018

The best results are highlighted in bold.

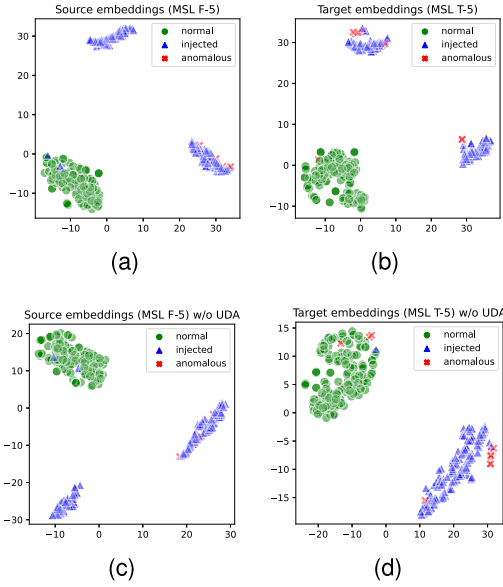


Fig. 3. Impact of UDA on DACAD feature representations. It contrasts the embeddings of (a) source entity F-5 with UDA, (b) target entity T-5 with UDA, (c) source entity F-5 without UDA, and (d) target entity T-5 without UDA.

Table IV offers several key insights into the impact of different loss components on the MSL dataset using F-5 as a source. Removing the target self-supervised CL (w/o $\mathcal{L}_{SelfCont}$) leads to lower metrics (F1: 0.481, AUPR: 0.495, AUROC: 0.697). Moreover, excluding source-supervised CL (w/o $\mathcal{L}_{SupCont}$) reduces effectiveness (F1: 0.463, AUPR: 0.427, AUROC: 0.639), highlighting its role in capturing source-specific features, which are crucial for the model's overall accuracy. Similarly, omitting the discriminator component results in performance reduction (F1: 0.471, AUPR: 0.484, AUROC: 0.699). However, the most significant decline occurs without the classifier (F1: 0.299, AUPR: 0.170, AUROC: 0.503), underscoring its crucial role in effectively distinguishing between normal/anomaly classes. Overall, the best results (F1: 0.595, AUPR: 0.554, AUROC: 0.787) are achieved with all components, highlighting the effectiveness of an integrated approach.

Overall, each component within the model plays a crucial role in enhancing its overall performance. The highest performance across all metrics (F1: 0.595, AUPR: 0.554, AUROC: 0.787) is achieved when all components are included.

To elucidate the effectiveness of UDA within DACAD, we examine the feature representations from the MSL dataset, as illustrated in Fig. 3. It presents the t-SNE 2D embeddings

TABLE V
EFFECT OF CEC CLASSIFIER ON MSL DATASET (SOURCE: F-5)

Classifier	F1	AUPR	AUROC
Our proposed CEC	0.595	0.554±0.268	0.787±0.268
BCE-based	0.467	0.504±0.304	0.670±0.241
DeepSVDD	0.428	0.400±0.250	0.595±0.185

The best results are highlighted in bold.

TABLE VI
EFFECT OF ANOMALY INJECTION ON MSL DATASET (SOURCE: F-5)

Approach	F1	AUPR	AUROC
with injection	0.595	0.554±0.268	0.787±0.268
w/o injection	0.489	0.461±0.276	0.682±0.666

The best results are highlighted in bold.

of DACAD feature representations $\phi^{CL}(\phi^R(w))$ for the MSL dataset. Each point represents a time series window, which can be normal, anomalous, or anomaly-injected. These representations highlight the domain discrepancies between source and target entities and demonstrate how DACAD aligns the time series window features effectively. The comparison of feature representations with and without UDA reveals a significant domain shift when UDA is not employed between the source (entity F-5) and the target (entity T-5) within the MSL dataset.

Effect of CEC: Table V compares the performance of our CEC classifier with two BCE and DeepSVDD - on the MSL dataset using F-5 as a source. Our proposed CEC shows superior performance compared to BCE and DeepSVDD across three metrics on the MSL dataset. With the highest F1 score, it demonstrates a better balance of precision and recall. Its leading performance in AUPR indicates greater effectiveness in identifying positive classes in imbalanced datasets. Additionally, CEC's higher AUROC suggests it is more capable of distinguishing between classes.

Effect of anomaly injection: We study the impact of anomaly injection in Table VI on the MSL dataset when using F-5 as a source. It shows that anomaly injection significantly improves all metrics (F1: 0.595, AUPR: 0.554, AUROC: 0.787), enhancing the model's ability to differentiate between normal and anomalous patterns, thereby improving DACAD's overall accuracy. Without anomaly injection, there is a notable decline in performance, emphasizing its role in precision. The higher standard deviation in AUROC scores without injection suggests more variability and less stability in the model performance. The study underscores the vital role of anomaly injection in improving anomaly detection models. It reveals that incorporating anomaly injection not only boosts detection accuracy but also enhances the model's overall stability.

VI. CONCLUSION

The DACAD model is an innovative approach to TSAD that is particularly effective in environments with limited labeled data. By combining domain adaptation and contrastive learning, it applies labeled anomaly data from one domain to detect anomalies in another. Its anomaly injection mechanism, introducing

a spectrum of synthetic anomalies, significantly bolsters the model's adaptability and robustness across various domains. Our evaluations on diverse real-world datasets establish DACAD's superiority in handling domain shifts and outperforming existing models. Its capability to generalize and accurately detect anomalies, regardless of the scarcity of labeled data in the target domain, represents a significant contribution to TSAD. In future work, we aim to refine the anomaly injection process further, enhancing the model's ability to simulate a broader range of anomalous patterns. Additionally, we plan to evolve the model to encompass univariate time series analysis, broadening its scope and utility. Furthermore, we intend to explore alternative feature extraction architectures beyond TCN to assess their impact on model performance.

REFERENCES

- [1] B. Xie, S. Li, F. Lv, C. H. Liu, G. Wang, and D. Wu, "A collaborative alignment framework of transferable knowledge extraction for unsupervised domain adaptation," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 7, pp. 6518–6533, Jul. 2023.
- [2] X. Liu et al., "Deep unsupervised domain adaptation: A review of recent advances and perspectives," *APSIPA Trans. Signal Inf. Process.*, vol. 11, no. 1, 2022.
- [3] H. I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: A review," *Data Mining Knowl. Discov.*, vol. 33, pp. 917–963, 2019.
- [4] Q. Wen et al., "Time series data augmentation for deep learning: A survey," 2020, *arXiv: 2002.12478*.
- [5] G. Wilson and D. J. Cook, "A survey of unsupervised deep domain adaptation," *ACM Trans. Intell. Syst. Technol.*, vol. 11, pp. 1–46, 2020.
- [6] H. Xu, Y. Wang, S. Jian, Q. Liao, Y. Wang, and G. Pang, "Calibrated one-class classification for unsupervised time series anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 5723–5736, Nov. 2024.
- [7] S. Purushotham, W. Carvalho, T. Nilanon, and Y. Liu, "Variational recurrent adversarial deep domain adaptation," in *Proc. Int. Conf. Learn. Representations*, 2016.
- [8] R. Cai et al., "Time series domain adaptation via sparse associative structure alignment," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 6859–6867.
- [9] Y. Ozyurt, S. Feuerriegel, and C. Zhang, "Contrastive learning for unsupervised domain adaptation of time series," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [10] G. Wilson, J. R. Doppa, and D. J. Cook, "Multi-source deep domain adaptation with weak supervision for time-series sensor data," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 1768–1778.
- [11] W. Zhang, L. Deng, L. Zhang, and D. Wu, "A survey on negative transfer," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 2, pp. 305–329, Feb. 2023.
- [12] Q. Zhou, S. He, H. Liu, J. Chen, and W. Meng, "Label-free multivariate time series anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3166–3179, Jul. 2024.
- [13] G. I. Quintana, L. Vancamberg, V. Jugnon, A. Desolneux, and M. Mougeot, "Bridging contrastive learning and domain adaptation: Theoretical perspective and practical application," 2025, *arXiv:2502.00052*.
- [14] K.-H. Lai et al., "Context-aware domain adaptation for time series anomaly detection," in *Proc. SIAM Int. Conf. Data Mining*, 2023, pp. 676–684.
- [15] Z. Z. Darban, G. I. Webb, S. Pan, C. C. Aggarwal, and M. Salehi, "CARLA: Self-supervised contrastive representation learning for time series anomaly detection," *Pattern Recognit.*, vol. 157, 2025, Art. no. 110874.
- [16] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [17] H. Li, W. Zheng, F. Tang, Y. Zhu, and J. Huang, "Few-shot time-series anomaly detection with unsupervised domain adaptation," *Inf. Sci.*, vol. 649, 2023, Art. no. 119610.
- [18] M. Long, Z. Cao, J. Wang, and M. I. Jordan, "Conditional adversarial domain adaptation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 1647–1657.
- [19] E. Tzeng, J. Hoffman, K. Saenko, and T. Darrell, "Adversarial discriminative domain adaptation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 7167–7176.
- [20] R. Zhao, Y. Xia, and Y. Zhang, "Unsupervised sleep staging system based on domain adaptation," *Biomed. Signal Process. Control*, vol. 69, 2021, Art. no. 102937.
- [21] Y. Yang, H. Zhang, and Y. Li, "Long-distance pipeline safety early warning: A distributed optical fiber sensing semi-supervised learning method," *IEEE Sensors J.*, vol. 21, no. 17, pp. 19453–19461, Sep. 2021.
- [22] G. Wang, M. Chen, Z. Ding, J. Li, H. Yang, and P. Zhang, "Inter-patient ECG arrhythmia heartbeat classification based on unsupervised domain adaptation," *Neurocomputing*, vol. 454, pp. 339–349, 2021.
- [23] N. Lu, H. Xiao, Y. Sun, M. Han, and Y. Wang, "A new method for intelligent fault diagnosis of machines based on unsupervised domain adaptation," *Neurocomputing*, vol. 427, pp. 96–109, 2021.
- [24] M. Ragab et al., "Contrastive adversarial domain adaptation for machine remaining useful life prediction," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5239–5249, Aug. 2021.
- [25] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, 1997.
- [26] J. Chung, K. Kastner, L. Dinh, K. Goel, A. C. Courville, and Y. Bengio, "A recurrent latent variable model for sequential data," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 2980–2988.
- [27] T. Zhou et al., "One fits all: Power general time series analysis by pretrained LM," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 43322–43355.
- [28] J. Liu et al., "Large language models can deliver accurate and interpretable time series anomaly detection," 2024, *arXiv:2405.15370*.
- [29] X. Zhang, R. R. Chowdhury, R. K. Gupta, and J. Shang, "Large language models for time series: A survey," 2024, *arXiv:2402.01801*.
- [30] M. Jin et al., "Large models for time series and spatio-temporal data: A survey and outlook," 2023, *arXiv:2310.10196*.
- [31] Y. Yang et al., "A survey on diffusion models for time series and spatio-temporal data," 2024, *arXiv:2404.18886*.
- [32] C. Wang et al., "Drift doesn't matter: Dynamic decomposition with diffusion reconstruction for unstable multivariate time series anomaly detection," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024, Art. no. 473.
- [33] P. Zhao, X. Wang, Y. Zhang, Y. Li, H. Wang, and Y. Yang, "Diffusion-UDA: Diffusion-based unsupervised domain adaptation for submersible fault diagnosis," *Electron. Lett.*, vol. 60, no. 3, 2024, Art. no. e13122.
- [34] S. Schmidl, P. Wenig, and T. Papenbrock, "Anomaly detection in time series: A comprehensive evaluation," *Proc. VLDB Endowment*, vol. 15, pp. 1779–1797, 2022.
- [35] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "Do deep neural networks contribute to multivariate time series anomaly detection?," *Pattern Recognit.*, vol. 132, 2022, Art. no. 108945.
- [36] H. Xu, G. Pang, Y. Wang, and Y. Wang, "Deep isolation forest for anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12591–12604, Dec. 2023.
- [37] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, "Deep learning for time series anomaly detection: A survey," *ACM Comput. Surv.*, vol. 57, no. 1, pp. 1–42, 2024.
- [38] Z. Yue et al., "TS2Vec: Towards universal representation of time series," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8980–8987.
- [39] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [40] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3395–3404.
- [41] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=LzQQ89U1qm_
- [42] Z. Niu, K. Yu, and X. Wu, "LSTM-based VAE-GAN for time-series anomaly detection," *Sensors*, vol. 20, 2020, Art. no. 3738.
- [43] J. Zhan, C. Wu, C. Yang, Q. Miao, and X. Ma, "HFN: Heterogeneous feature network for multivariate time series anomaly detection," *Inf. Sci.*, vol. 670, 2024, Art. no. 120626.
- [44] F. Zhou, G. Wang, K. Zhang, S. Liu, and T. Zhong, "Semi-supervised anomaly detection via neural process," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10423–10435, Oct. 2023.
- [45] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2828–2837.

- [46] H. Deng, Y. Sun, M. Qiu, C. Zhou, and Z. Chen, "Graph neural network-based anomaly detection in multivariate time series data," in *Proc. IEEE Annu. Comput. Softw. Appl. Conf.*, 2021, pp. 1128–1133.
- [47] L. Shen, Z. Li, and J. Kwok, "Timeseries anomaly detection using temporal hierarchical one-class network," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 13016–13026.
- [48] Q. Lei, J. Yi, R. Vaculin, L. Wu, and I. S. Dhillon, "Similarity preserving representation learning for time series clustering," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 2845–2851.
- [49] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2021, pp. 2114–2124.
- [50] S. Tonekaboni, D. Eytan, and A. Goldenberg, "Unsupervised representation learning for time series with temporal neighborhood coding," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [51] H. Zhou, K. Yu, X. Zhang, G. Wu, and A. Yazidi, "Contrastive autoencoder for anomaly detection in multivariate time series," *Inf. Sci.*, vol. 610, pp. 266–280, 2022.
- [52] Y. Yang, C. Zhang, T. Zhou, Q. Wen, and L. Sun, "DCdetector: Dual attention contrastive representation learning for time series anomaly detection," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2023, pp. 3033–3045.
- [53] Z. Zhong, Z. Yu, Y. Yang, W. Wang, and K. Yang, "PatchAD: A lightweight patch-based MLP-mixer for time series anomaly detection," 2024, *arXiv:2401.09793*.
- [54] Y. Fang, J. Xie, Y. Zhao, L. Chen, Y. Gao, and K. Zheng, "Temporal-frequency masked autoencoders for time series anomaly detection," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 1228–1241.
- [55] D. Lee, S. Malacarne, and E. Aune, "Explainable time series anomaly detection using masked latent generative modeling," *Pattern Recognit.*, vol. 156, 2024, Art. no. 110826.
- [56] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 815–823.
- [57] P. Khosla et al., "Supervised contrastive learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 18661–18673.
- [58] C. Lea, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks: A unified approach to action segmentation," in *Proc. Eur. Conf. Comput. Vis. Workshops*, Springer, 2016, pp. 47–54.
- [59] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath, "Generative adversarial networks: An overview," *IEEE Signal Process. Mag.*, vol. 35, no. 1, pp. 53–65, Jan. 2018.
- [60] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by back-propagation," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1180–1189.
- [61] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Rev.*, vol. 60, no. 2, pp. 223–311, 2018.
- [62] S. Ghadimi and G. Lan, "Stochastic first-and zeroth-order methods for nonconvex stochastic programming," *SIAM J. Optim.*, vol. 23, no. 4, pp. 2341–2368, 2013.
- [63] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," in *Proc. MLSDA 2nd Workshop Mach. Learn. Sensory Data Anal.*, 2014, pp. 4–11.
- [64] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "LSTM-based encoder-decoder for multi-sensor anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1724–1732.
- [65] E. Tzeng, J. Hoffman, N. Zhang, K. Saenko, and T. Darrell, "Deep domain confusion: Maximizing for domain invariance," 2014, *arXiv:1412.3474*.
- [66] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, "Towards a rigorous evaluation of time-series anomaly detection," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 7194–7201.
- [67] A. Huet, J. M. Navarro, and D. Rossi, "Local evaluation of time series anomaly detection algorithms," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 635–645.
- [68] K.-H. Lai, D. Zha, J. Xu, Y. Zhao, G. Wang, and X. Hu, "Revisiting time series outlier detection: Definitions and benchmarks," in *Proc. 35th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track*, 2021.

Zahra Zamanzadeh Darban is a PhD researcher with the Department of Data Science and AI, Monash University, Australia. She focuses on anomaly detection in time series using deep learning. Supervised by dr. Salehi, professor pan, and professor webb, she previously spent seven years as a software engineer, machine learning engineer, and system analyst in the industry before joining Monash.

Yiyuan Yang is currently working toward the DPhil degree with the Department of Computer Science, University of Oxford, U.K. He focuses on the field of intelligent sensing systems, time series, spatiotemporal data mining, anomaly detection, and generative models. He previously studied with the Department of Automation, Tsinghua University and interned with the Alibaba DAMO Academy and Huawei Noah's Ark Lab.

Geoffrey I. Webb (Fellow, IEEE) is the research director with Monash University Data Futures Institute, Australia. Former editor-in-chief of DMKD (2005-2014), he chaired ACM SIGKDD and IEEE ICDM conferences. Advisor to BigML Inc and FROOMLE, he innovated in association discovery and rule search. Awards include the 2017 Eureka Prize in data science.

Charu C. Aggarwal (Fellow, IEEE) received the BS degree from IIT Kanpur, in 1993, and PhD degree from Massachusetts Institute of Technology, in 1996. He is a research scientist at the IBM T.J. Watson Research Center, working in performance analysis, databases, and data mining. He has served as program vice chair for SDM 2007, ICDM 2007, WWW 2009, and ICDM 2009, and was associate editor of IEEE TKDE (2004-2008). He currently serves on editorial boards of TKDD, DMKD, SIGKDD Explorations, and KAIS. He is also a Fellow of the ACM.

Qingsong Wen received the PhD in electrical and computer engineering from Georgia Tech. He is the head of AI Research and chief scientist at Squirrel Ai Learning. He has published 100+ papers, including Oral/Spotlight papers at NeurIPS, ICML, and ICLR, and won awards such as IJCAI Most Influential Paper and AAAI IAAI Deployed Application Award. He serves as associate editor for IEEE TPAMI, IEEE SPL, and Neurocomputing, and guest editor for Applied Energy and IEEE IoT Journal. His research focuses on AI for time series, education, and general machine learning.

Shirui Pan received the PhD degree in computer science from UTS and is a professor at Griffith University. His research in data mining and machine learning has appeared in top venues like Nature Machine Intelligence, KDD, and ICLR. He has received several awards, including the 2024 IEEE CIS TNNLS Outstanding Paper Award, 2020 ICDM Best Student Paper, and 2024 IEEE ICDM Tao Li Award, and is an ARC Future Fellow.

Mahsa Salehi received the PhD degree in computer science from the University of Melbourne, Australia, in 2016. She then joined IBM Research as a postdoctoral researcher. In 2017, she joined Monash University, Faculty of IT, where she is currently a senior lecturer. Her research includes time series analytics and anomaly detection. She serves as an associate editor of the *Transactions on Knowledge Discovery from Data*.